.SYSTEMX: Portable Project Coordination for AI-Assisted Work

White-paper edition 2.1 | Software baseline: 1.11.0-beta.1

Prepared 31 October 2026 by Lucas (SatoshiUNO) for Wayne Tech Lab LLC.

Source at beta tag | Manual | Historical edition 2.0

Abstract

`.SYSTEMX` is a portable record and coordination layer for AI-assisted projects. It keeps an objective, master plan, canonical task ledger, selected memory, and verification evidence in a versioned project folder. The beta template adds Agent Beta for named BOT/DOT workspaces, Agent 1 for advisory security and access review, and Agent Alpha's Agenda for selected context settings. A default-deny Access Matrix catalog now gives each built-in role and instance pattern a standard entry. These are project records and local tools; they do not create a trusted identity, grant a provider permission, install an operating-system service, or deploy an application. The public repository is a reusable template. Large authoring files, including Blender projects, belong in ignored or separately managed external data, with published assets and provenance kept in the template.

This paper explains the current record model, agent responsibilities, access and secret boundaries, release accounting, portable data, and an evaluation method for efficiency. It is an implementation and release-boundary account, not an independent security audit, malware-clean certificate, cross-platform acceptance report, or measured token-savings study. Edition 2.0 remains an immutable historical paper with its own alpha.4 technical baseline and checksums.

1. One coordination layer, distinct authorities

The exact hidden directory name is `.SYSTEMX`. An adopted root keeps purpose and constraints in `GLOBAL/CONTEXT.md`, outcomes in `PLAN/MASTER-PLAN.md`, canonical task states in `WORK/TASKS.json`, and accepted decisions or resumable checkpoints in selected memory. A registered child project keeps its own records under `Projects/NAME/.SYSTEMXP` while sharing reviewed outer tools. The public template begins with blank project facts. Setup and updates are additive: a managed release keeps its own immutable defaults, previews conflicts, and preserves adopter-owned records.

Each independently accepted unit of work receives a stable task ID and explicit acceptance condition. The six task states are `todo`, `in_progress`, `blocked`, `needs_review`, `done`, and `cancelled`. Generated views and harness TODO lists are projections or external observations; they do not replace the canonical ledger. `done` depends on review against the task's criteria, not just an exit code or a worker's completed flag. Optional ROOT links request provenance and planning levels to observed harness or bot items using explicit bindings; its read-only delta identifies differences for an authorized actor to reconcile.

Agent 0 or the user owns acceptance. Host tools and provider accounts enforce permissions. A role label, bot name, or Access Matrix row cannot grant access. Native subagents, CLIs, and online assistants retain separate identities and readback.

Agent X records event time and due observations; it does not execute a scheduler. Agent Y handles an explicitly delegated installed CLI and a bounded branch/PR lane; a worktree does not isolate its operating-system or network access. Agent Z computes a repeatable score from supplied answers and evidence references; it does not perform every underlying check. Agent Alpha selects among ten AUTOBOT reference files on demand, rather than starting ten model processes. These roles supplement Agent 0 without displacing canonical task acceptance.

Agent Beta (`agent.b`) registers named workspaces under `AGENTB/.SYSTEMXB-BOT_NAME` with nonsecret metadata and a selected source such as `local` or `online-reference`. It does not contact the named service. Agent 1's 400-question catalog is separate from Agent Z's 100-question review policy.

Agenda is an optional Agent Alpha settings table. Its template `options` array is empty. It may reference selected provider rule files and reviewed Access Matrix decisions when an owner explicitly records those choices; it does not schedule work, run AUTOBOTs, or change provider instructions. The ten AUTOBOT files remain fixed reference functions, not ten background workers. Selected-scope `session-settings` gives a short status and rule fingerprint so a session can load only the relevant source files. Its compact form preserves an aggregate rule count, absent IDs, and a revision digest; the digest detects local change, not external truth or permission.

2. Access Matrix: complete role coverage, no implicit grant

`GLOBAL/ACCESS-MATRIX.json` is the machine-readable selected-scope record. The distributed `GLOBAL/ACCESS-MATRIX.xlsx` is a human-reviewable projection. The standard catalog has 20 rows: seven named agents (0, 1, Alpha, Beta, X, Y, Z); one generic numeric-worker pattern; one host-native subagent pattern; one named Agent Beta BOT/DOT instance pattern; and ten fixed AUTOBOT reference roles. Every standard row has `templateGrant: none` and `requiresReview: true`. A new project may inherit this catalog while its actual `decisions` list remains empty. No account name, credential, project authorization, or person-specific grant is seeded into the public template.

An applied access decision is a separate, selected-scope `ACC-...` record with an actor, resource, action, environment, owner, evidence, and recheck context. The format can represent reviewed intent, denial, or an unresolved gap. It cannot enforce GitHub IAM, operating-system ACLs, an IDE's tool policy, or a cloud provider's authorization. A workbook edit alone is not an active grant: validate and reconcile the canonical JSON before treating a row as current. A bot workspace may cite a decision ID, but a cited row still does not authenticate the bot or open a provider connection.

`agenda decision-add` previews one scoped decision and requires a matching revision plus `--apply` to write it. `agenda workbook-export` likewise requires preview, `--expected-revision`, and `--apply` to write a deterministic, formula-free view under ignored `local/`. `agenda workbook-check` compares it with canonical JSON. Neither workbook grants access.

3. Security review and secret delivery

Agent 1 (`agent.1`, when that numeric ID is free) defaults to warning-only `all-in-one` review. Its catalog contains 20 categories of 20 questions each and ten event buckets. The local check can inventory dependency manifest names and `.env` variable names without returning values. Unanswered questions remain unanswered; a filename scan is not a 400-question assessment. Device malware, spyware, keylogger, and MFA-interception posture are bounded signals requiring appropriate external evidence. Unknown and untested states must not be reported as pass.

The `.env` migration path is intentionally staged: inspect names, create a names-only `.env.example` if authorized, select each secret's destination, verify the target identity, and then use a hidden terminal prompt for an explicitly applied `gh secret set` operation. Newly issued credentials are the preferred migration path; an existing credential can be treated as a bounded test choice and later rotated. A GitHub Actions secret is a workflow input, not a local secret store; it cannot be symlinked back into a local `.env`. A local runtime requires a separately approved OS credential store or interactive input. `gh secret list` can confirm name and scope metadata but cannot verify the stored value or the consumer's behavior. The original `.env` and historical logs are preserved until the owner verifies consumers and retention obligations.

Agent 1's optional session helper is an explicitly started, bounded JSON-lines stdio process with read-only `ping`, `capabilities`, `assess`, and `stop` operations. It opens no port and installs no daemon. The implementation does not provide a macOS launchd unit, Linux systemd unit, Windows service, or IDE integration. A successful protocol response is not a security verdict. Repository writer access can also be a path to Actions secrets through workflow changes; a `.SYSTEMX` guest label does not prevent that. Actual repository and environment controls must be checked at action time. GitHub's Actions secret guidance describes the provider's scopes and CLI behavior.

4. Release identity, logs, and deployment accounting

The beta continues the published `1.11.0-alpha.N` line as `1.11.0-beta.1`; resetting it to `0.0.01` would obscure the existing release history. The public tag is `v1.11.0-beta.1`, and Python package metadata uses `1.11.0b1`. `VERSION`, `SOURCE.json`, package metadata, and the exact-file distribution manifest must agree. The manifest records SHA-256 hashes for the portable template files. Published tags and releases are immutable identities; later fixes need a new version rather than moving a tag. These conventions follow Semantic Versioning 2.0.0 and the Python packaging version specification.

A release counter counts published software release records under a stated selection rule and query time. A deployment counter counts observed deployment records or verified environment changes, not Git commits, tags, packages, screenshots, or successful local tests. The template may be released without creating a hosted application deployment. GitHub's deployment history associates a deployment with environment, commit, status, and possibly a URL; provider-specific deployments require their own receipts. Keep release, local validation, package, provider, installed-host, and owner-acceptance states separately in logs and release notes.

5. Template and external-data boundary

The public repository distributes portable defaults, selected images, documentation, code, tests, and a file inventory. It must not ship a real project registry, account, secret, absolute private-drive path, owner payload, raw render cache, or editable Blender project. The existing historical four-page field-guide JPEGs remain linked to their alpha baseline. Beta visual assets are separately versioned and described with accessible text; their presence does not imply that a service, daemon, or bot ran. Authoring archives may live in a user-selected external location. A path reference is valid only when that location is actually present in the selected environment and an authorized user can access it.

The package and managed-release archive have finite size limits. Publishing large authoring files inside the portable bundle would increase installation cost and can exceed those limits. Derived media outside the package needs a release asset identity and checksum, while selected small template graphics may be included in the manifest. A Git ignore rule helps prevent accidental inclusion but is not proof that material was never committed or synchronized elsewhere.

6. Input efficiency and what remains unmeasured

The intended efficiency path is to read selected scope, a bounded task/context packet, the few relevant rule files, and evidence on demand. A `session-settings --compact` response omits repetitive per-rule hashes from the full output and replaces them with a single aggregate revision. A development sample of the blank template returned 3,248 bytes in the full form and 1,043 bytes in the compact form, a 67.9% output-byte reduction for that command. The exact byte count is sample-specific and can change with records or formatting. It is not a measured reduction in billed input tokens, total latency, total cost, or accepted-work effort.

To evaluate real savings, compare representative tasks with the same model, tools, acceptance criteria, and cache conditions. Record all workers' provider-reported input, cached input, output, tool calls, retries, elapsed time, and accepted outcome. Include the cost of creating and maintaining the handoff. A shorter packet that omits a necessary constraint is a regression, regardless of its byte count. The original paper's token model remains illustrative.

7. Beta acceptance boundary

The beta release can be accepted as a tested reusable source distribution when its selected-scope setup, validator, managed-update path, build, bundled files, workbook/JSON parity, and public links are verified against the same source revision. That is a source-and-package result. It is not a claim that every adopter's GitHub, local secret store, OS service manager, provider account, dependency graph, or device was checked. On this release path, the Agent 1 service remains manual stdio; Windows and Linux host integration and real secret-consumer workflows require separate tests. The 400-question instrument starts with zero answered questions. No malware-clean or anti-interception guarantee follows from the catalog.

The next operational step for an adopter is to initialize a selected project, review its actual accounts and resources, record narrowly scoped access decisions, then test the provider and runtime that will consume them. Keep warnings advisory unless the project's owner adopts a stricter policy. Preserve prior release, log, and white-paper identities as historical evidence. Edition 2.0 and its PDF still describe alpha.4; this edition is the current beta paper with its explicit limits.

Source notes

1. WayneTechLab, `.SYSTEMX` beta source at its release tag, especially `VERSION`, `SOURCE.json`, `config/distribution.json`, `GLOBAL/ACCESS-MATRIX.json`, `scripts/agent_beta.py`, `scripts/security_agent1.py`, `scripts/agenda_access.py`, `scripts/session_service.py`, and `scripts/session_settings.py`. The software version and distribution manifest identify the published source; this paper does not claim independent validation.

2. WayneTechLab, `.SYSTEMX` edition 2.0 white paper, prepared 20 October 2026 and examined against `1.11.0-alpha.4`; its metadata and exact-byte checksums remain in that immutable release.

3. Semantic Versioning 2.0.0, prerelease precedence and immutable published version requirements.

4. Python Packaging Authority, Version specifiers, Python prerelease spelling.

5. GitHub, Using secrets in GitHub Actions and Viewing deployment history, provider-specific boundaries.